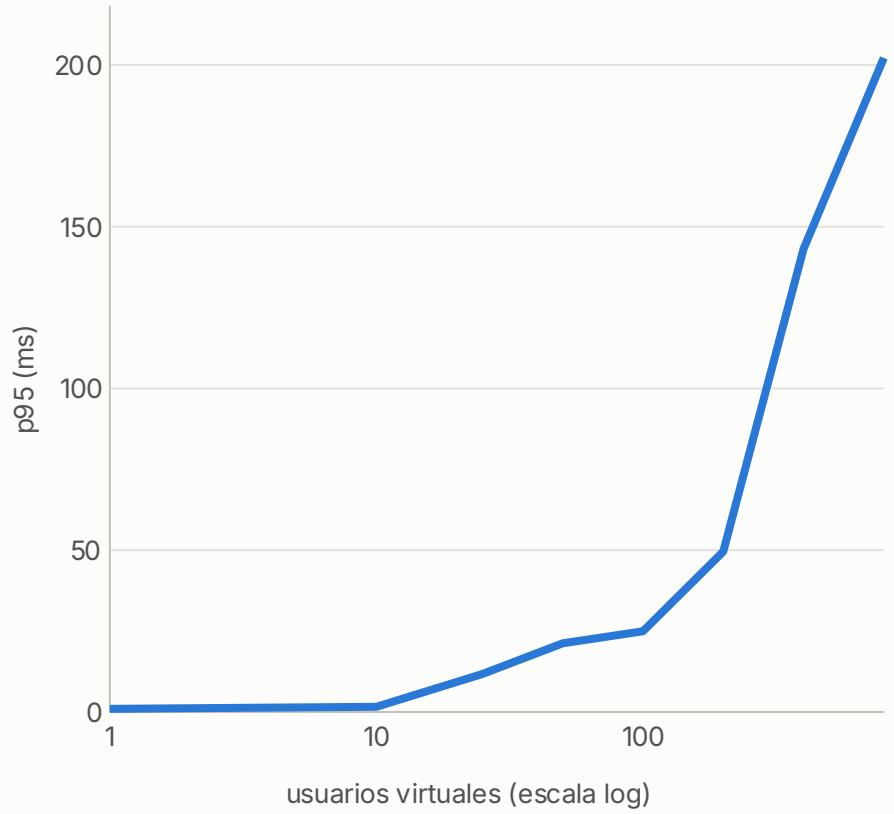
TESTING Y VALIDACIÓN DE SOFTWARE

Pruebas de carga y rendimiento

Cómo medir, leer y explicar el comportamiento de un servicio cuando muchos usuarios llegan a la vez.



Medición real de este taller: el p95 se mantiene plano y de pronto se dispara.

Taller: Registraduría (Spring Boot + H2) medida con k6

César Augusto Vega Fernández · Universidad de La Sabana

AGENDA

La ruta de hoy

1

El vocabulario

Anatomía de una petición, tipos de prueba y por qué se habla de p95 y no de promedio.

2

Qué pasa al saturar

Curvas reales: throughput, latencia, cola, modelo abierto vs. cerrado y un pico.

3

Escribir la prueba

Reglas del script, el falso verde, errores comunes y herramientas.

4

Del síntoma a la causa

Observabilidad, gates en CI y cómo madura un equipo en rendimiento.

Anatomía de una petición

Latencia

Tiempo que la petición pasa viajando por la red.

Tiempo de procesamiento

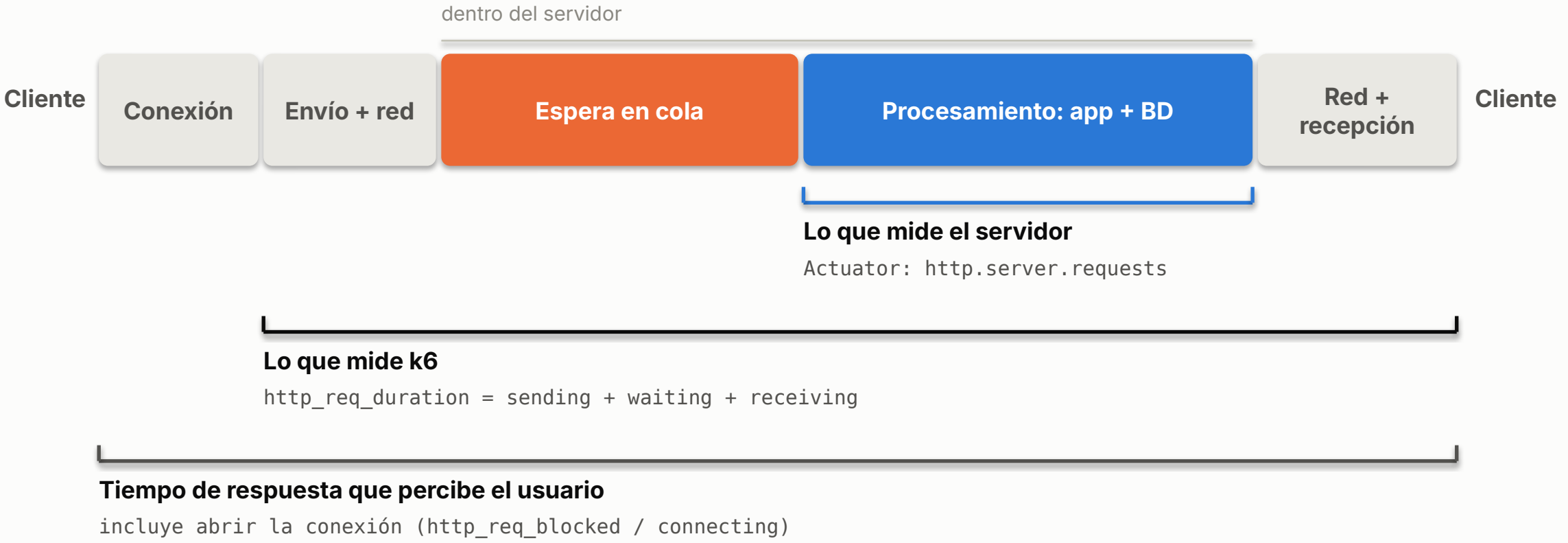
Lo que tardan la aplicación y la base de datos.

Tiempo de respuesta

Todo el recorrido, ida y vuelta, cola incluida.

Throughput

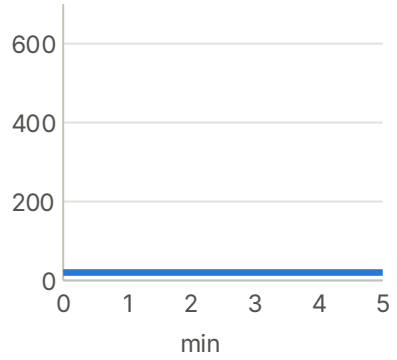
Peticiones completadas por segundo. No es un tramo: es una tasa.



Cinco preguntas, cinco perfiles de carga

Línea base

`SCENARIO=baseline`

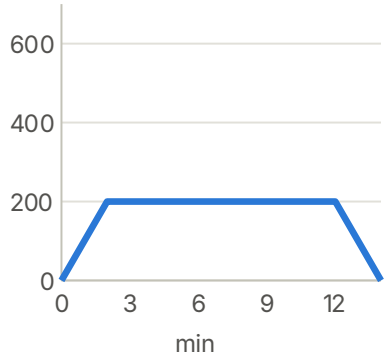


20 VUs · 5 min

¿Cuál es nuestra referencia para comparar?

Carga

`SCENARIO=load`

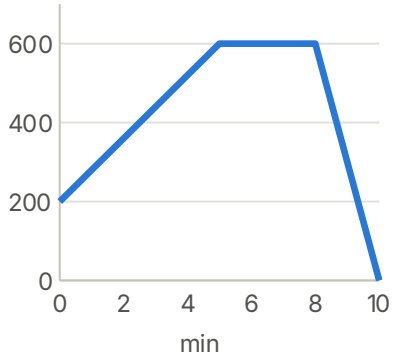


0→200 VUs · 14 min

¿Cumplimos el SLO con la demanda esperada?

Estrés

`SCENARIO=stress`

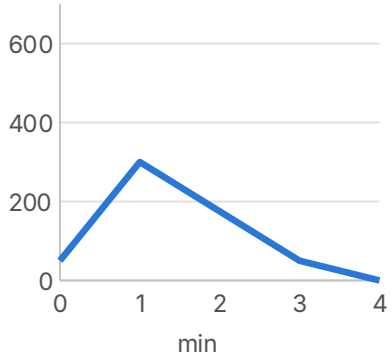


200→600 VUs · 10 min

¿Dónde se rompe y cómo se degrada?

Pico

`SCENARIO=spike`

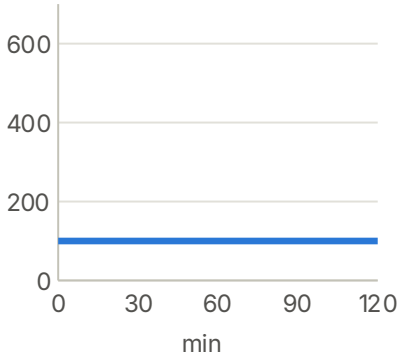


50→300→50 VUs · 4 min

¿Se recupera tras un salto brusco?

Resistencia

`SCENARIO=soak`



100 VUs · 2 h

¿Se degrada con las horas? Fugas, GC, conexiones.

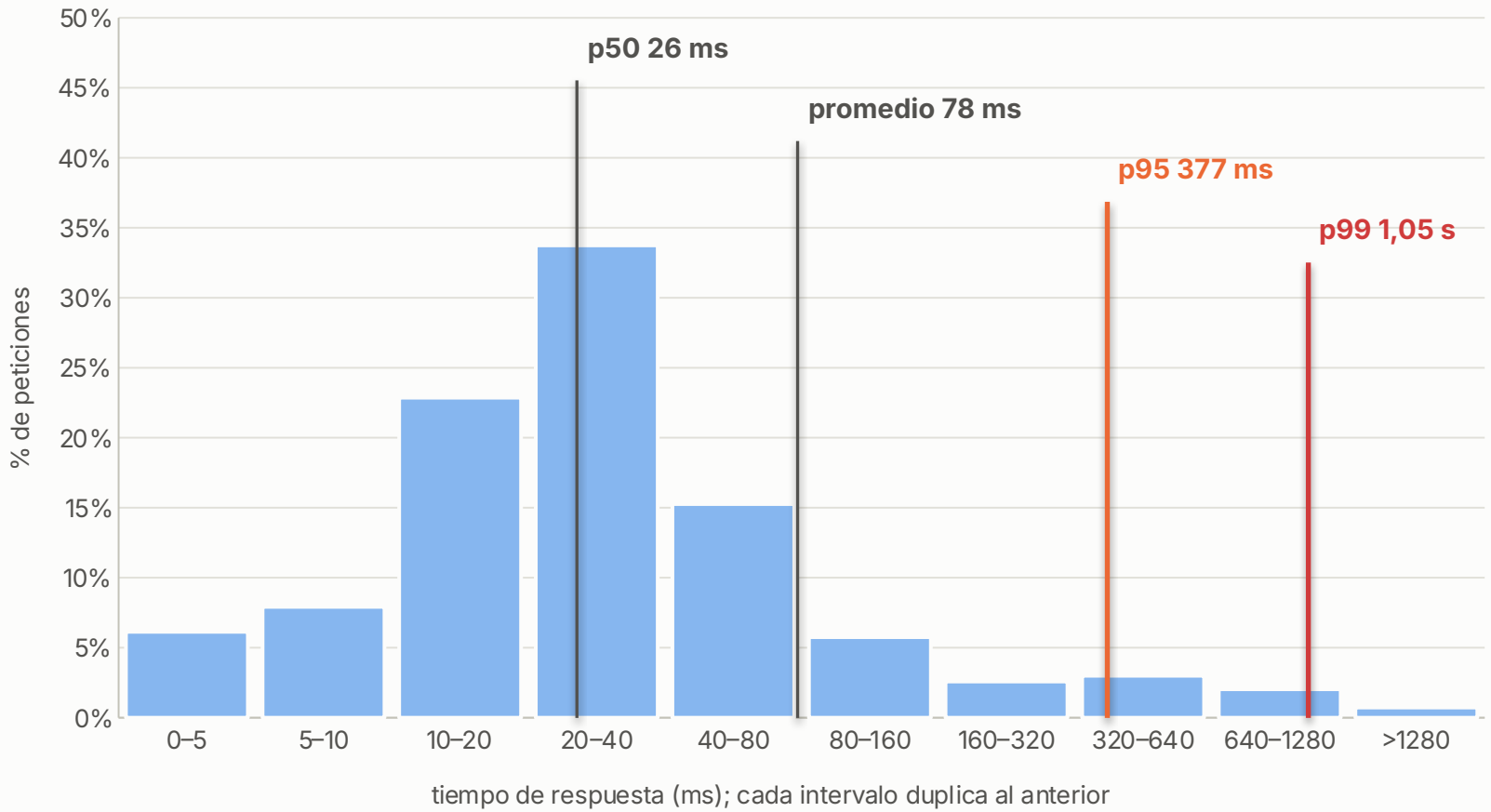
Eje vertical: usuarios virtuales, con la misma escala en las cinco tarjetas para poder compararlas.

Carga, estrés, pico y resistencia no son lo mismo

	Carga	Estrés	Pico	Resistencia
Pregunta	¿Cumple el SLO con la demanda esperada?	¿Dónde está el límite y cómo falla?	¿Aguanta un salto brusco y se recupera?	¿Se degrada con el paso de las horas?
Perfil	Rampa realista hasta el nivel esperado	Escalones por encima de lo esperado	Salto de ×5 a ×10 en segundos	Carga normal, constante
Duración	Minutos a una hora	Hasta encontrar el quiebre	Pocos minutos	Horas (2 h en el taller)
Qué mirar	p95/p99 y tasa de error contra el SLO	Punto de quiebre y tipo de fallo	Tiempo de recuperación	Memoria, GC, conexiones abiertas
Alarma	p95 > 300 ms o errores > 1 %	Errores que no bajan al quitar carga	Latencia que no vuelve a la base	Memoria que sube y no baja

BLOQUE 1 · PERCENTILES

Promedio: 78 ms. El 1 % más lento: más de 1,05 s.



Cómo leerlo

Cada barra es la fracción de peticiones que tardó ese tiempo. Las barras de la derecha son los usuarios con peor suerte.

p50 **26 ms**
la mitad tardó menos

p95 **377 ms**
95 de cada 100 tardaron menos

p99 **1,05 s**
99 de cada 100 tardaron menos

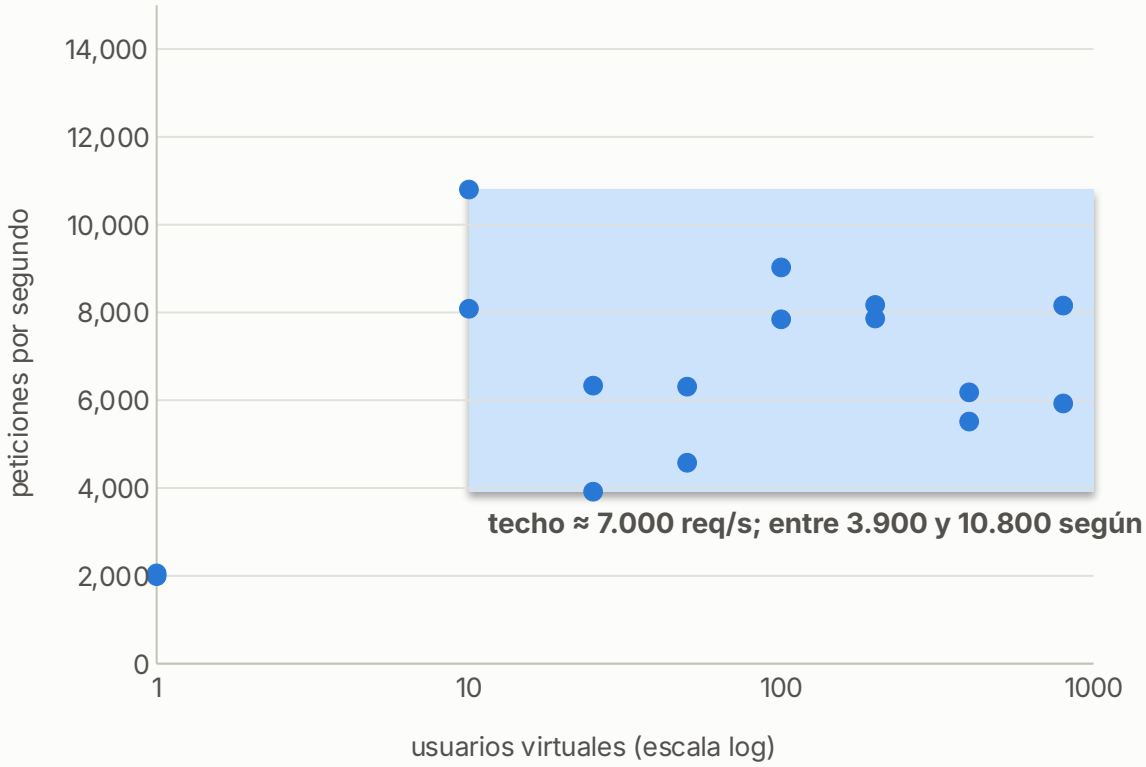
El p99 es 13 veces el promedio.

Fuente: 34.776 peticiones, 100 VUs, 30 s, código sin pool. k6 1.3 contra la registraduría del taller (Spring Boot 2.7 + H2 en memoria), portátil i7-13620H. Servicio fijado a 1 núcleo y k6 a otros 14 hilos, para que el cuello de botella fuera el servidor y no el inyector.

Más usuarios no es más throughput: es más espera

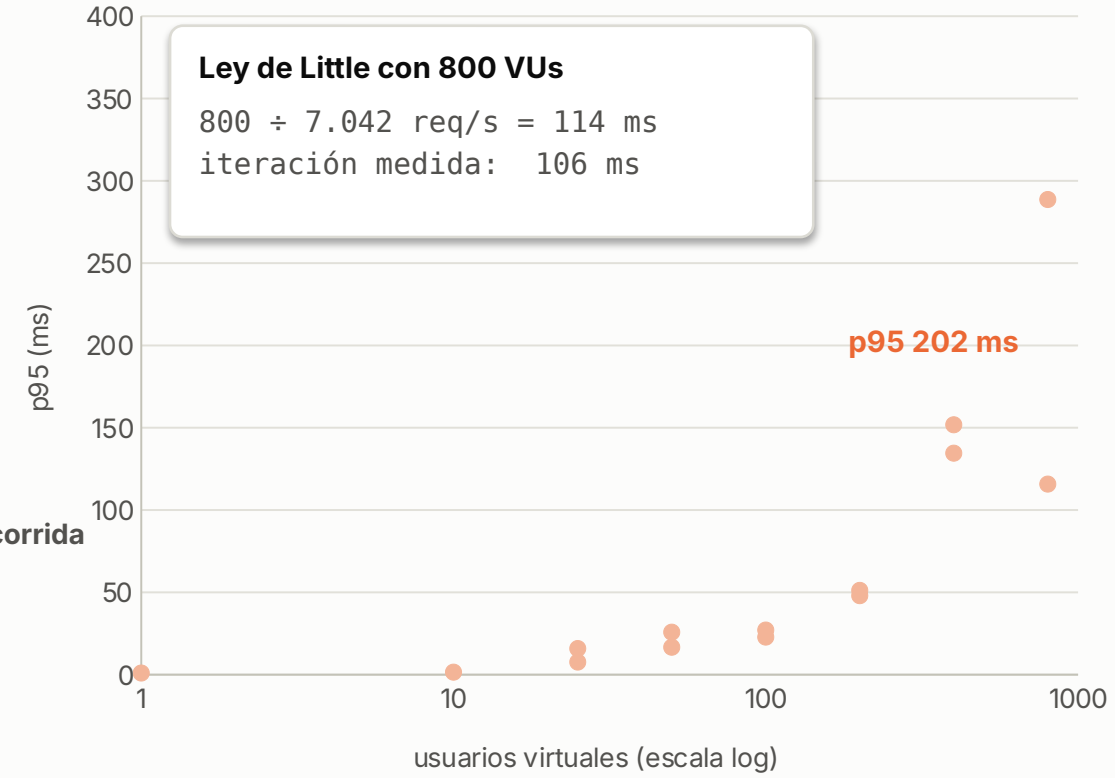
Throughput

puntos: cada corrida · franja: rango entre corridas desde 10 VUs



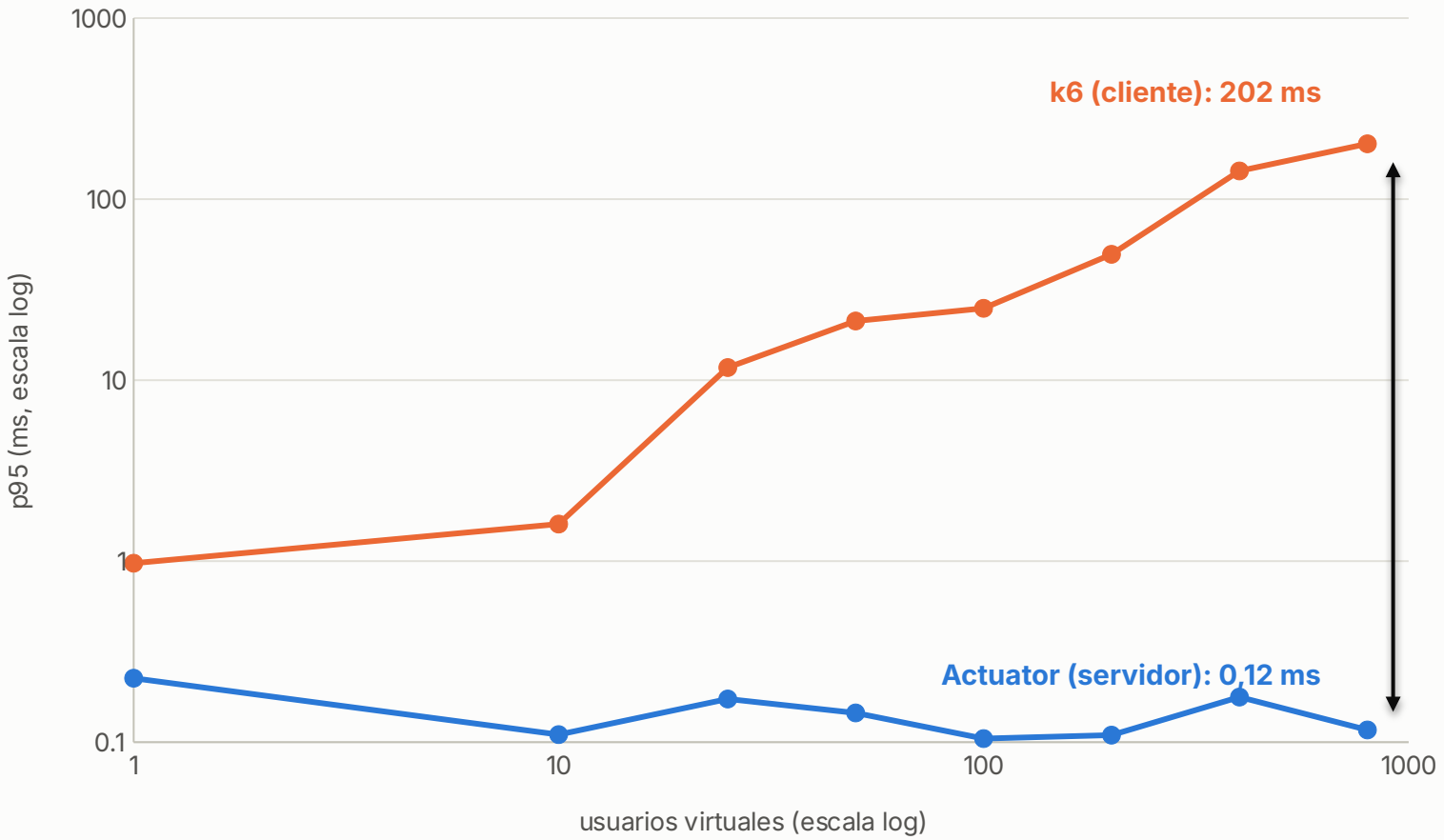
Tiempo de respuesta (p95)

línea: promedio · puntos claros: cada corrida



Fuente: Servicio con pool, 20 s por escalón, dos corridas. k6 1.3 contra la registraduría del taller (Spring Boot 2.7 + H2 en memoria), portátil i7-13620H. Servicio fijado a 1 núcleo y k6 a otros 14 hilos, para que el cuello de botella fuera el servidor y no el inyector.

Servidor rápido, usuario esperando: los dos tienen razón



202 ms vs. 0,12 ms

Con 800 VUs, el p95 del cliente fue más de mil veces el del servidor.

¿Dónde está la diferencia?

En la red, en el propio cliente y sobre todo en la cola: peticiones que ya llegaron y esperan un hilo libre. Actuator empieza a medir cuando salen de ahí.

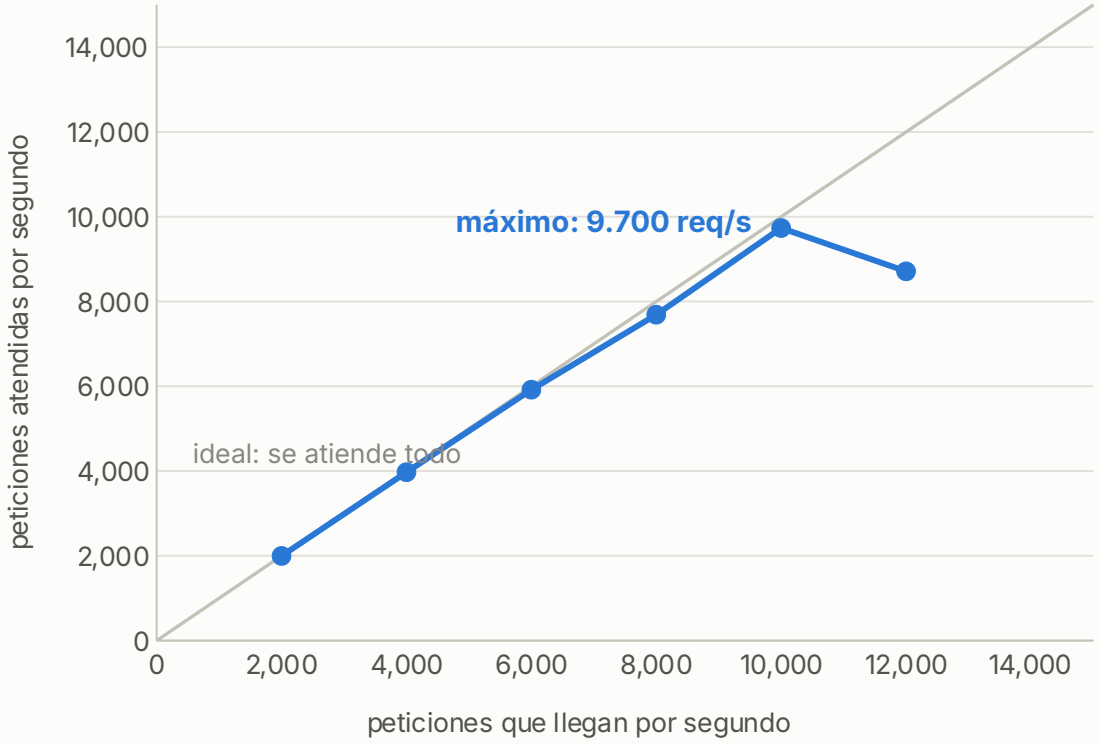
Qué hacer con esto

No tocar el código de negocio: aumentar capacidad o reducir el trabajo por petición.

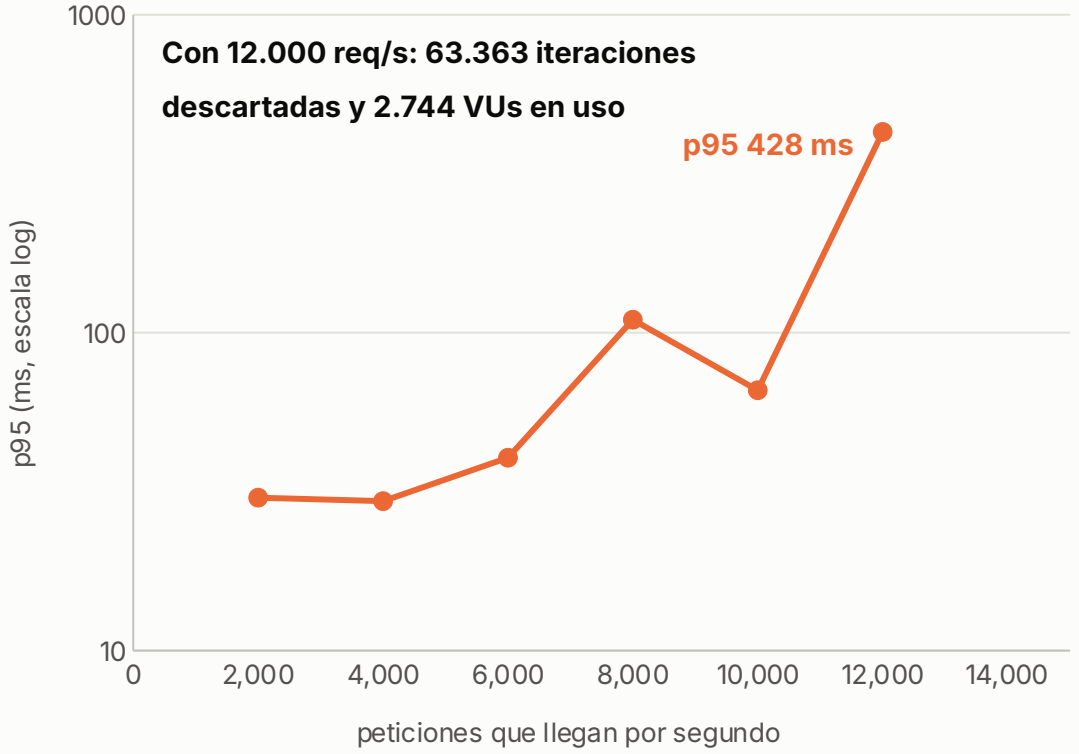
Fuente: Servicio con pool. k6 1.3 contra la registraduría del taller (Spring Boot 2.7 + H2 en memoria), portátil i7-13620H. Servicio fijado a 1 núcleo y k6 a otros 14 hilos, para que el cuello de botella fuera el servidor y no el inyector.

Modelo abierto: el tráfico no espera a que el servidor termine

Lo que llega vs. lo que se atiende



La latencia al pasar el límite



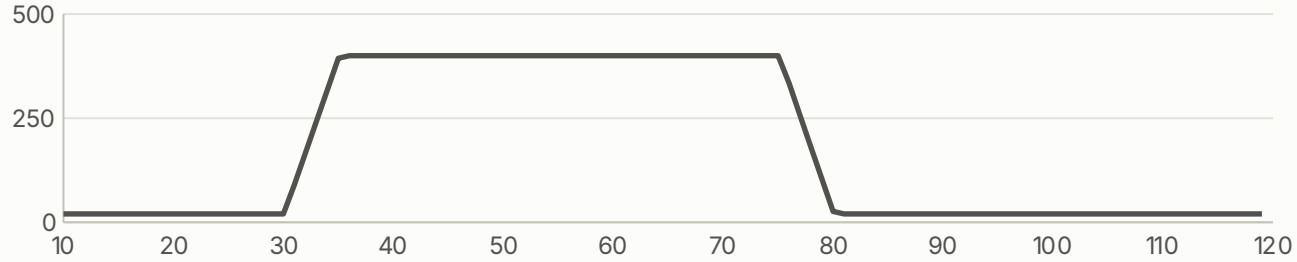
Mismo experimento con el código sin pool: con 2.000 req/s llegando atendió solo 491–1.171 y falló el 5–20 % de las peticiones.

Fuente: constant-arrival-rate, 20 s por escalón, promedio de dos corridas. k6 1.3 contra la registraduría del taller (Spring Boot 2.7 + H2 en memoria), portátil i7-13620H. Servicio fijado a 1 núcleo y k6 a otros 14 hilos, para que el cuello de botella fuera el servidor y no el inyector.

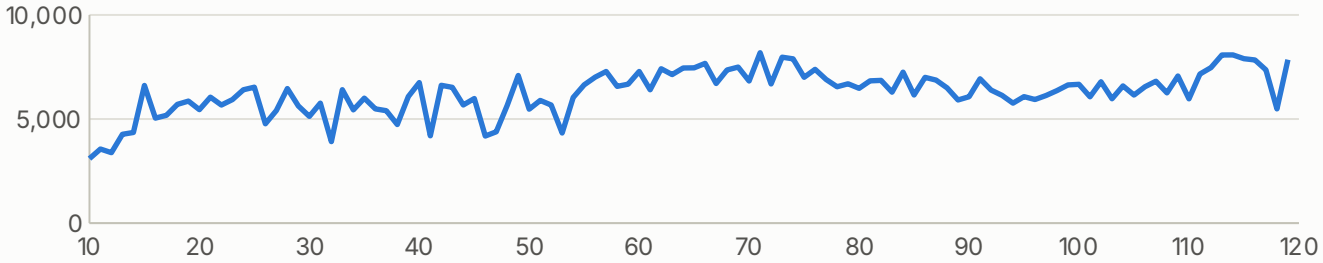
BLOQUE 2 · PRUEBA DE PICO

Un pico, segundo a segundo

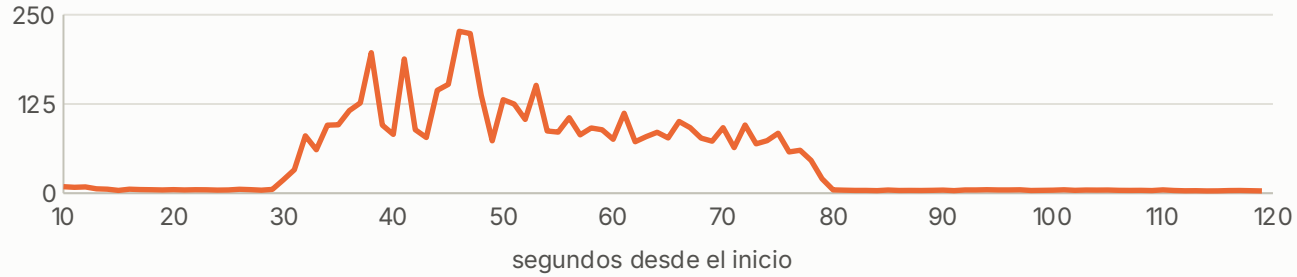
Usuarios virtuales



Peticiones por segundo



p95 por segundo (ms)



MEDIANA POR TRAMO

antes del pico

p95 **4,9 ms**

5.449 req/s

durante el pico

p95 **91 ms**

6.675 req/s

después

p95 **4,0 ms**

6.551 req/s

Fuente: ramping-vus con --out csv, agregado por segundo, código sin pool. k6 1.3 contra la registraduría del taller (Spring Boot 2.7 + H2 en memoria), portátil i7-13620H. Servicio fijado a 1 núcleo y k6 a otros 14 hilos, para que el cuello de botella fuera el servidor y no el inyector.

Cuatro reglas para que el script se parezca a la realidad

01 Validar el contenido, no solo el código

Un 200 puede traer un rechazo de negocio. Compare el cuerpo con el resultado esperado.

```
outcome === v.expected
```

02 Datos variados desde un CSV

Repetir el mismo registro calienta cachés y dispara reglas de duplicado. SharedArray carga el CSV una vez para todos los VUs.

```
new SharedArray('voters', () =>
parse(open(csv)))
```

03 Pausas de usuario (think time)

Un usuario lee antes de hacer clic. Sin pausas, pocos VUs generan una carga que nadie produciría.

```
sleep(SLEEP_MS / 1000)   // 100 ms por
defecto
```

04 Gestionar el estado de la prueba

Ids únicos por VU e iteración. Entre corridas, reinicie el servicio o desplace el rango.

```
ID_BASE + __VU * 1000000 + (__ITER %
1000000)
```

Un 200 OK no significa que funcionó

Respuestas reales de POST /register

200 OK	**VALID**	persona viva, mayor de edad, id nuevo
200 OK	**UNDERAGE**	edad de 0 a 17
200 OK	**DEAD**	alive = false
200 OK	**INVALID_AGE**	edad negativa o mayor de 120
200 OK	**DUPLICATED**	el id ya existe

Mismo código HTTP, cinco resultados de negocio distintos. Solo uno es un registro exitoso.

Tres formas de validar, de peor a mejor

Falso verde

```
check(res, { 'ok': (r) => r.status === 200 });
```

Trampa sutil

```
'body VALID': () => body.includes('VALID')
// 'INVALID_AGE'.includes('VALID') === true
```

Correcto

```
'resultado de negocio esperado': () =>
    outcome === v.expected
// expected viene de voters.csv
```

register_voter_k6.js ya usa la tercera forma, con la métrica register_failed.

Errores comunes y cómo reconocerlos

ERROR	CÓMO SE VE	CÓMO EVITARLO
Inyector sobrecargado	La máquina que genera la carga se queda sin CPU o red y los tiempos salen inflados. **Visto al preparar esta clase: CPU del portátil al 99 %; k6 reportaba p95 de 8,3 ms y el servidor menos de 1 ms.**	Monitorear también el inyector; separar CPUs o máquinas.
Estado de prueba sucio	Una segunda corrida reusa ids ya registrados y todo sale DUPLICATED.	Reiniciar el servicio o usar ID_BASE entre corridas.
Probar solo al final	El cuello de botella aparece en preproducción, cuando cambiar el diseño es caro.	Escenario corto en cada PR (shift-left).
Ignorar errores intermitentes	Un 0,2 % de fallos "de vez en cuando" suele ser un pool agotado o un timeout.	Leer el tipo de error en k6 y el log del servidor, no solo la tasa.
Aleatoriedad pura	Con pocas iteraciones, el azar no reparte bien los casos y dos corridas no se comparan.	Recorrer los datos barajados en orden, como un mazo de cartas.

BLOQUE 3 · ERRORES REALES

Lo que salió mal al medir para esta clase

El inyector compitiendo con el servicio

Antes: k6 y el servicio en los mismos 16 hilos: CPU al 99 %.

Después: servicio en 1 núcleo, k6 en otros 14 hilos.

Sin separarlos, parte del tiempo que reporta k6 es k6 esperando CPU.

La JVM no sabía cuántos núcleos tenía

Antes: p99 del servidor de hasta 476 ms con 100 VUs.

Después: menos de 1 ms con -XX:ActiveProcessorCount.

Una JVM que cree tener 16 CPUs pero corre en 2 dimensiona mal sus hilos de GC. Además no compartía núcleo con k6.

Medir sin calentar

Antes: 3.370 req/s en la primera corrida tras arrancar.

Después: 6.420 req/s en la siguiente, idéntica.

El compilador JIT todavía estaba trabajando. Siempre hay un calentamiento que no se mide.

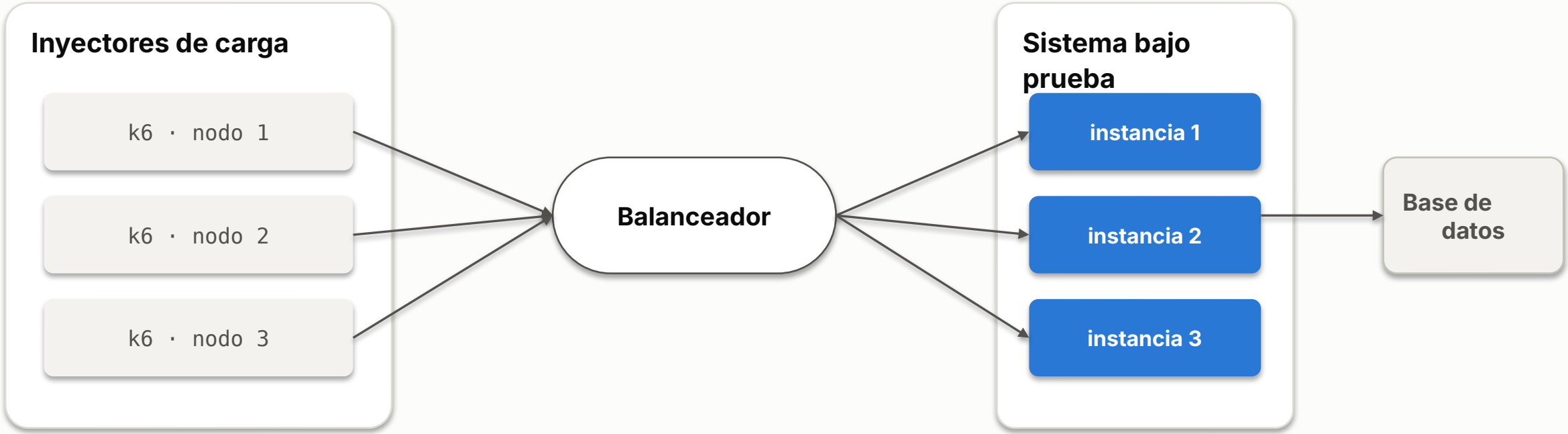
Buscar el error solo en el log

Síntoma: 10 % de peticiones fallidas con 400 VUs.

En el log: ni una sola excepción en el log del servidor.

Bajo saturación, muchos fallos ocurren antes de llegar a la aplicación: conexiones y timeouts. Hay que leer los errores en k6.

Cuando un solo inyector no alcanza



Observabilidad de TODOS los nodos: CPU y red de los inyectores, métricas del servicio, base de datos

El inyector nunca es el cuello de botella

Si su CPU pasa del ~80 %, los tiempos que reporta ya no son del servicio.

Pocas IPs de origen

Un balanceo por IP puede mandar toda la carga a una sola instancia.

Rampa con tiempo para escalar

El autoescalado reacciona en minutos; una rampa de segundos lo prueba mal.

Herramientas: se elige por el equipo, no por moda

Herramienta	Cómo se escribe la prueba	Fortaleza	Buena elección cuando…
k6	JavaScript; motor en Go	Scripts versionables, thresholds como gate de CI	el equipo vive en el repositorio y en CI
Apache JMeter	Plan .jmx con interfaz gráfica	Muchos protocolos y plugins; muy conocido	se prueban protocolos variados o hay equipos QA sin código
Gatling	DSL en Java, Kotlin o Scala	Alta concurrencia por nodo; reporte HTML	el equipo es JVM y necesita mucho volumen
Locust	Python	Usuarios modelados como clases; UI web en vivo	el comportamiento del usuario es complejo
Artillery	YAML + JavaScript (Node.js)	Configuración corta; HTTP, WebSocket	se quiere algo liviano en un proyecto Node

Resaltada: la herramienta del taller. Los conceptos (VUs, rampas, percentiles, umbrales) son los mismos en todas.

BLOQUE 4 · DIAGNÓSTICO

Del síntoma a la causa en tres niveles

1 **¿Qué ve el usuario?**

El síntoma

k6: p95 / p99, http_req_failed, register_failed, peticiones por segundo

2 **¿Dónde se va el tiempo?**

La ubicación

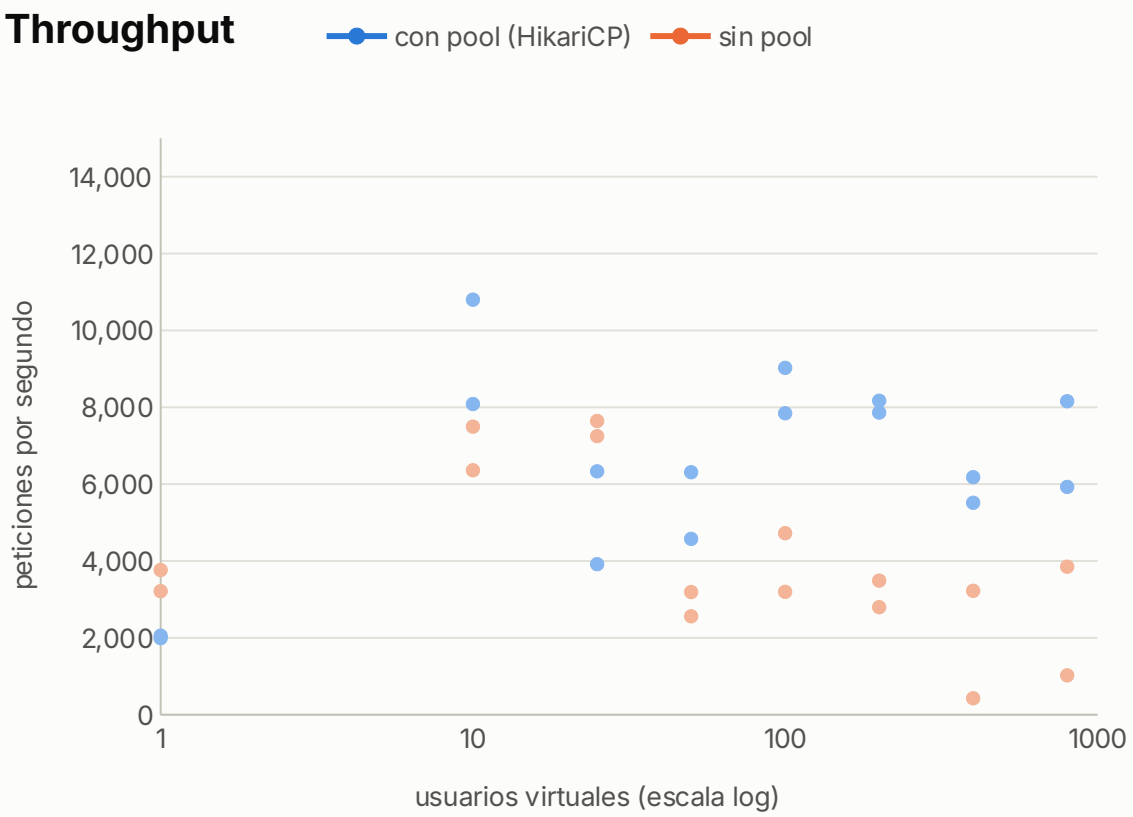
Actuator: http.server.requests. Si el servidor dice 1 ms y k6 dice 90 ms, el tiempo está en la cola, no en el código. Con varios servicios: trazas distribuidas (OpenTelemetry).

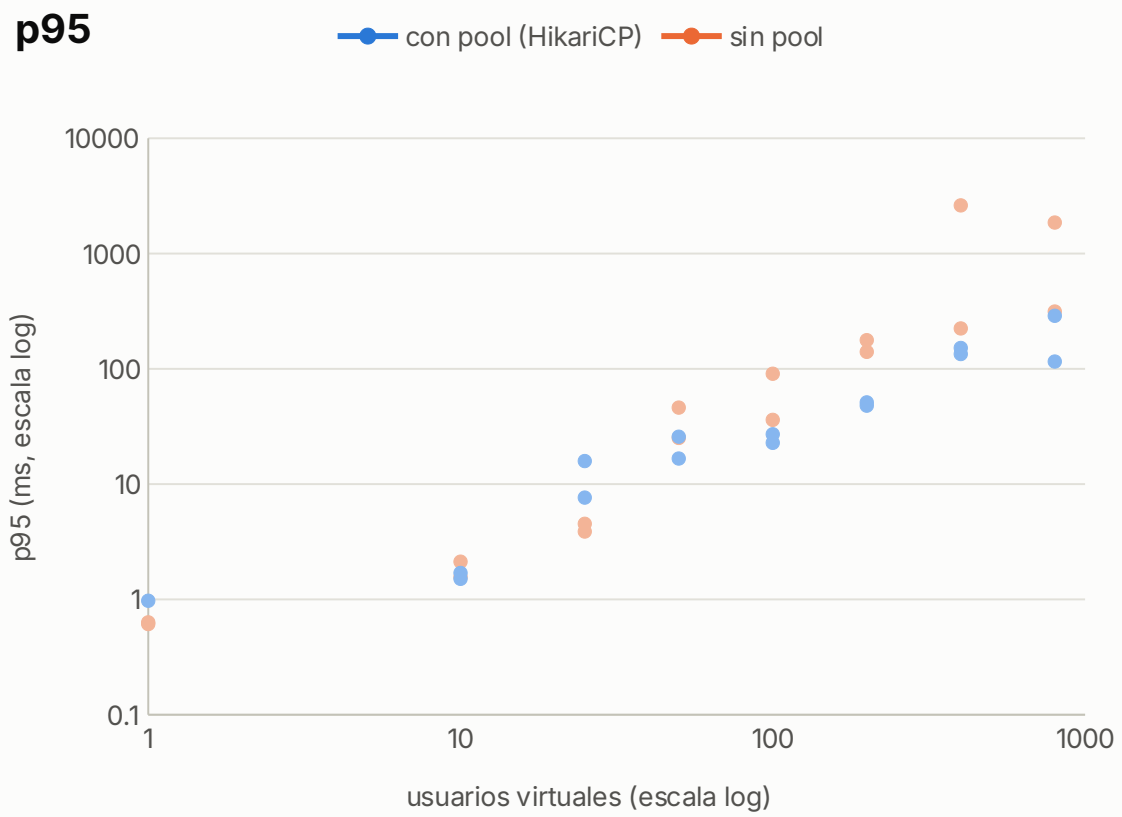
3 **¿Por qué?**

La causa raíz

tomcat.threads.busy, jvm.threads.live, jvm.gc.pause, CPU, conexiones del pool. Causas típicas: conexión nueva por petición, pool agotado, fuga de memoria, consulta sin índice.

Sin pool de conexiones: un defecto que solo aparece con carga

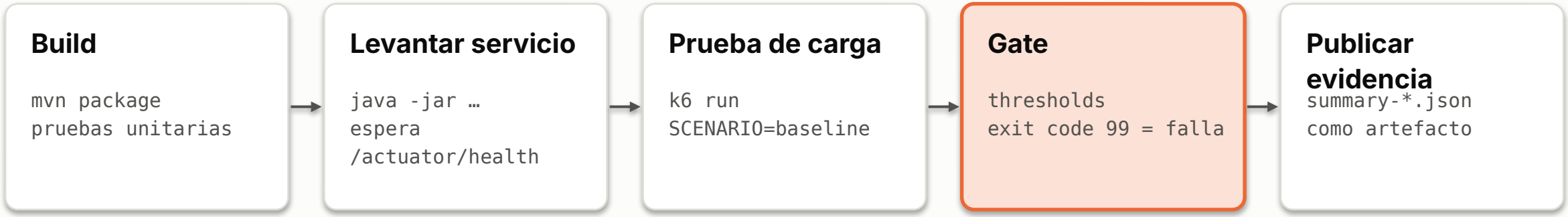




Hasta 25 VUs, sin diferencia clara. Desde 50, sin pool atiende del 31 al 53 % de lo que atiende con pool, y su peor corrida falló el 28 %.

Fuente: Mismo código salvo el pool, 20 s por escalón, dos corridas (puntos claros). k6 1.3 contra la registraduría del taller (Spring Boot 2.7 + H2 en memoria), portátil i7-13620H. Servicio fijado a 1 núcleo y k6 a otros 14 hilos, para que el cuello de botella fuera el servidor y no el inyector.

El rendimiento como gate del pipeline




```
thresholds: {
  http_req_failed: ['rate<0.01'],
  'http_req_duration{status:200}':
      ['p(95)<300', 'p(99)<800'],
  register_failed: ['rate<0.01'],
}
```

Tomado de perf/scripts/register_voter_k6.js

El umbral es el SLO escrito en código

Si el p95 pasa de 300 ms o los errores del 1 %, k6 sale con código 99 y el paso falla solo.

Corto en cada PR, largo de noche

load dura 14 minutos y soak dos horas: bloquearían la revisión.

El pipeline levanta el servicio

Y espera a /actuator/health con un bucle, no con un sleep fijo.

BLOQUE 4 · MADUREZ

¿En qué nivel está su equipo?

Al terminar el taller, un equipo debería estar en el nivel 3.

1

Reactivo

Se prueba cuando algo falla en producción. Herramienta básica, sin escenarios ni SLO.

2

Scripting realista

Datos variados, validación de negocio, pausas, estado de prueba controlado.

3

Continuo

Escenario corto en cada PR con gate; los largos, programados. Regresiones detectadas antes de producción.

4

Guiado por SLO

Observabilidad en producción, presupuesto de error y experimentos de caos para validar la recuperación.

El rendimiento no se agrega al final. Se diseña, se mide y se vigila.

Para entregar en este taller

- Baseline, carga y estrés ejecutados, con su summary-*.json
- Matriz de rendimiento: escenario, SLO, resultado
- p95 de k6 comparado con el p95 de Actuator, y la explicación de la diferencia
- Antes y después de agregar un pool de conexiones
- Pipeline con gate y al menos un defecto documentado en defectos.md